# 1 概述【Overview】

模组测温结果受环反射温度，大气温度，大气湿度（绝对湿度），目标发射率，大气透过率，目标距离等因素影响。因此想要获得准确的测温结果，需要对模组进行环境变量矫正。

【The temperature measurement of the module is affected by Emissivity, Atmospheric transmissivity, ambient reflection temperature, ambient atmospheric temperature, target distance, etc.】

# 2 影响测温的环境变量【Ambient variables】

- 反射温度：周围物体发出的红外线，会被目标物体反射。

  【Ambient reflection temperature：Target objects reflect thermal radiation from their surroundings.】

- 大气温度，大气湿度（绝对湿度），大气透过率：大气会吸收目标物体的红外辐射，同时自身也会产生辐射。

  【Ambient atmospheric temperature，Atmospheric humidity，Atmospheric transmissivity：The atmosphere absorbs infrared radiation from the target and produces radiation of its own.】

- 目标发射率：实际物体与理想黑体辐射红外的能力的比值。

  【Emissivity：The ratio of an actual object's ability to emit infrared radiation to an ideal blackbody.】

- 目标距离：由于一些复杂的光学、结构问题，同样的目标物在不同的距离下，测得的温度也不相同。

  【Distance: Due to some complex optical and structural problems, the temperature of the same object measured at different distances is not the same.】

> 非专业客户，可以认为反射温度和大气温度相同；大气湿度采用默认湿度；
>
> 【For non-professional customers, reflection temperature and atmospheric temperature can be considered the same; Atmospheric humidity adopts default humidity;】

# 3 等效大气透过率【LUT Equivalent atmospheric transmissivity LUT】

为了方便用户使用，我们将大气温度、大气湿度、大气透过率、目标距离等环境因素整合为等效大气透过率，并提供一份默认LUT。用户只需要输入大气温度、大气湿度、目标距离，即可通过该LUT，获取等效大气透过率，输入模组进行矫正。如果用户的散热结构和光学结构与我司内部测试用差距较大，可能需要用户自己标定生成该LUT，我们将提供计算生成该LUT的软件工具。

【For the convenience of users, we integrate atmospheric temperature, atmospheric humidity, atmospheric transmissivity, distance and other ambient factors into equivalent atmospheric transmissivity, and provide a default LUT. Users only need to input atmospheric temperature, atmospheric humidity and distance to obtain the equivalent atmospheric transmissivity through the LUT, and input the module for correction. If the heat dissipation structure and optical

structure of the user are quite different from those of our company's internal test, the user may need to calibrate and generate the LUT by himself. We will provide software tools to calculate and generate the LUT. 】

# 3.1 LUT介绍【LUT instructions】

目前LUT表的格式是用三维数组表示不同大气湿度，大气温度和目标距离的等效透过率。

【The current LUT table is to use a THREE-DIMENSIONAL array to represent the equivalent transmissivity of different atmospheric humidity, atmospheric temperature and distance. 】

- 大气湿度共分4个等级

  【Atmospheric humidity--- 4 levels】
- 大气温度共有14个温度点

  【The atmospheric temperature--- 14 points 】
- 目标距离共有64个距离

  【Distance--- 64 points】

# 3.2 大气湿度等级【humidity level】

划分区间待定（暂无接口），当前实验条件有限，4个湿度等级的等效大气透过率是相同的。

【The division interval is to be determined (no interface is available), the current experimental conditions are limited, and the equivalent atmospheric **transmissivity** of the four humidity levels is the same. 】

# 3.3 温度点如下：(单位/℃)【atmospheric temperature(℃)】

-5,0,5,10,15,20,25,30,35,40,45,50,55,55，最后一个是为客户预留的温度点

【The last one is reserved for the customer】

# 3.4 距离点如下：(单位/m)【distance（℃）】

0.25, 0.30, 0.35, 0.40, 0.45, 0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95, 1.00, 1.05, 1.10, 1.15, 1.20, 1.30, 1.40, 1.50, 1.60, 1.70, 1.80, 1.90, 2.00, 2.20, 2.40, 2.60, 2.80, 3.00, 3.20, 3.40, 3.60, 3.80, 4.00, 4.50, 5.00, 5.50, 6.00, 6.50, 7.00, 7.50, 8.00, 9.00, 10.00, 11.00, 12.00, 13.00, 14.00, 16.00, 18.00, 20.00, 22.00, 24.00, 26.00, 28.00, 30.00, 35.00, 40.00, 45.00, 50.00。

# 3.5 表格的存储方式如下【The table is stored as follows】

LUT存储成一个bin文件，按unsigned short存储，方便各平台调用。目前默认表格是存放在`src/main/assets`目录下的`tau_H.bin`和`tau_L.bin`

【The LUT is stored as a bin file, unsigned short, which is easy to call on all platforms. The current default tables are `tau_H.bin` and `tau_L.bin` in the `src/main/assets` directory 】

若用Tau[i][j][k]代表第i个湿度点，第j个温度点和第k个距离点下(从0开始计数)的等效大气透过率，该数值是bin文件中第(i*64*14+j*64+k)*个数，分别读取bin文件中(i*64*14+j*64+k)*2和(i*64*14+j*64+k)*2+1位置的2个byte，按小端存放方式解析出来的数值即为经过14位量化后的等效大气透过率。

【If Tau[i][j][k] represent the value of the i-th humidity point, the j-th Atmospheric temperature point and the k-th distance point (counting from 0). This value is the number of (i *64*14+j*64+k) in bin file. Read two bytes in the positions of (i *64*14+j*64+k)*2 and (i *64*14+j*64+ k)*2+1 respectively. The value resolved according to the little-end storage mode is the Tau after 14-bit quantization. 】

# 4 环境变量修正【Ambient variable correct】

> 环境变量修正，可通过固件或者SDK来实现，推荐使用SDK来实现
>
> 【Ambient variable correct can be implemented using firmware or SDK. SDK is recommended 】

## 4.1 固件实现FW solution

### 4.1.1 Step1

根据大气温度，大气湿度，距离查表等效大气透过率【According to atmospheric temperature, atmospheric humidity, distance lookup table equivalent atmospheric transmissivity 】

如上所述表格的格式，通过大气湿度判断选用哪张表，如果当前温度在第i和i+1个温度点之间，目标距离在第j和j+1个距离点之间，可先读取bin文件中温度点和距离点分别为 (i,j),( i,j+1), (i+1,j), (i+1,j+1)位置的等效大气透过率，进行双线性插值。也可选择最近的温度点和距离点读取等效大气透过率，作为近似值。

【According to the format of the table above, which table to choose is judged by atmospheric humidity. If the current temperature is between the i and i +1 temperature points, and the target distance is between the j and j +1 distance points, The equivalent atmospheric transmissivity of temperature point and distance point are (i, j),(i+1,j), (i+1,j) and (i+1,j+1) in bin file can be read first for bilinear interpolation. The nearest temperature and distance points can also be selected to read the equivalent atmospheric transmissivity as an approximation. 】

### 4.1.2 Step2

输入目标发射率，大气透过率，大气温度，反射温度。【Input emissivity, atmospheric transmissivity, atmospheric temperature, and reflection temperature. 】

通过`IRCMD`库中的`setPropTPDParams`函数，分别设置：(其中读取的大气透过率需转换成7位量化后的数值)

【Through the `setPropTPDParams` function in `IRCMD` library, set the values respectively :(the atmospheric transmissivity read should be converted into 7-bit quantized value) 】

- 目标发射率emissivity： TPD_PROP_EMS，value(7位量化7-bit，取值范围0-128)
- 大气透过率transmissivity： TPD_PROP_TAU，value(7位量化7-bit，取值范围0-128)
- 大气温度atmospheric temperature： TPD_PROP_TA，开尔文温度Kevin(高增益High Gain：230-430；低增益Low Gain 230-873)
- 反射温度reflection temperature： TPD_PROP_TU，开尔文温度Kevin (高增益High Gain：230-430；低增益Low Gain230-873)
- 存在问题：精度较低，针对整张图像中的所有场景都采用同一套环境变量参数，对不同的目标测温精度不同。

【Problems: Low accuracy. The same set of ambient variable parameters are used for all scenes in the whole image, and the temperature measurement accuracy is different for different targets. 】

## 4.2 SDK库实现【SDK solution】

SDK的demo中的，首先读取模组中相关的测温参数和环境变量参数，根据大气温度，距离，湿度自动推算等效大气透过率，然后进行测温的环境变量校正，计算过程精度较高，并且不同的目标可以采用不同的环境校正参数校正，因此测温精度较高。

【demo： first read module in relevant temperature measurement parameters and ambient variables, according to the atmospheric temperature, distance, humidity automatically calculates the equivalent atmospheric transmissivity, and the ambient variable correction of temperature measurement, calculation accuracy is higher, and different goals can be used in a different ambient correction parameter correction, so temperature measurement precision is higher. 】

### 4.2.1 Step1

根据不同的高低增益读取对应的`tau_x.bin`文件，并调用函数`readNucTableFromFlash`获取机芯中的测温修正系数。该方法比较耗时，需要在子线程中执行，且只需要执行一次，和`releaseTemperatureCorrection`方法成对出现。

【Read the corresponding `tau_x.bin` file according to different high and low gains, and call the function `readNucTableFromFlash` to obtain the temperature measurement correction coefficient in the movement. This method is time-consuming, needs to be executed in a child thread, and only needs to be executed once, and it appears in pairs with the `releaseTemperatureCorrection` method.】

```
/**
 * 获取机芯中的测温修正系数【read Nonuniformity correction table form sensor flash】
 * <p>
 * {@link #temperatureCorrection(double, byte[], double, double, double, double, double, long)}
 * {@link #releaseTemperatureCorrection(long)}
 * <p>
 * 该方法比较耗时，需要在子线程中执行，且只需要在初始化的时候执行一次。
 * 【This method is time-consuming, needs to be executed in a child thread, and only needs to be executed once during initialization.】
 * 在退出的时候需要执行{@link #releaseTemperatureCorrection(long)}释放资源。
 * 【When exiting, you need to execute {@link #releaseTemperatureCorrection(long)} to release resources.】
 *
 * @return the handle for {@link #temperatureCorrection(double, byte[], double, double, double, double, double, long)} and
 * {@link #releaseTemperatureCorrection(long)}
 */
public long readNucTableFromFlash()
```

```
...
private long tempinfo = 0; // 机芯中的测温修正系数
...

```

```
new Thread(new Runnable() {
    @Override
    public void run() {
        AssetManager am = mContext.getAssets();
        InputStream is;
        try {
            // 根据不同的高低增益加载不同的文件
            int[] value =
ircmd.getPropTPDParams(CommonParams.TPD_PROP_GAIN_SEL);
            Log.d(TAG, "TPD_PROP_GAIN_SEL=" + value[0]);
            if (value[0] == 1) {
                // 当前机芯为高增益
                is = am.open("tau_H.bin");
            } else {
                // 当前机芯为低增益
                is = am.open("tau_L.bin");
            }
            int lenth = is.available();
            tau_data = new byte[lenth];
            if (is.read(tau_data) != lenth) {
                Log.d(TAG, "read file fail ");
            }
            Log.d(TAG, "read file lenth " + lenth);
        } catch (IOException e) {
            e.printStackTrace();
        }
        // 获取机芯中的测温修正系数
        tempinfo = ircmd.readNucTableFromFlash();
        //
        Message message = new Message();
        message.what = MESSAGE_CODE_READ_NUC_SUCCESS;
        mHandler.sendMessage(message);
    }
}).start();
```

该函数只需要在初始化的时候执行一次，如果有高低增益的切换，则该函数需要重新调用一次，即需要在切换之后重新读取对应的增益表。

【This function only needs to be executed once during initialization. If there is a switch between high and low gain, the function needs to be called again, that is, the corresponding gain table needs to be re-read after the switch.】

## 4.2.2 Step2

设定新的目标发射率，大气温度，反射温度，距离，湿度进行新的测温修正系数计算，传入原始的摄氏度温度，调用`temperatureCorrection`函数进行测温修正，输出修正后的摄氏度温度。

【Set the new target emissivity, atmospheric temperature, reflected temperature, distance, and humidity to calculate a new temperature measurement correction coefficient, input the original temperature in degrees Celsius, call the `temperatureCorrection` function to perform temperature measurement correction, and output the corrected temperature in degrees Celsius.】

```
/**
```

```
     * 计算新的环境变量校正参数【Calculate new environment variable correction parameters】
     * correct the tempertuatrue by software
     * The function must call after {@link #readNucTableFromFlash()}
     * and release the resource called {@link #releaseTemperatureCorrection(long)}
     *
     * @param temp          旧的温度(单位:摄氏度)【Celsius degree】
     * @param taudata       存放大气透过率LUT的表格【the calibration parameters】
     * @param ems           目标发射率(0-1)【the object emissivity ,the value should be from 0.0 to 1.0】
     * @param ta            大气温度(单位:摄氏度)【the atmospheric temperature in Celsius degree】
     * @param tu            反射温度(单位:摄氏度)【the object reflection temperature in Celsius degree】
     * @param dist          目标距离(0-50,单位:m)【the distance between the object and detector,value should be from 0.0 to 50.0】
     * @param hum           环境相对湿度(0-1)【the object humidity ,the value should be from 0.0 to 1.0】
     * @param temp_cal_info the handle created by {@link #readNucTableFromFlash()}
     * @return new temperature value 新的温度(单位:摄氏度)【Celsius degree】
     */
    public double temperatureCorrection(double temp, byte[] taudata, double ems, double ta, double tu,
                                        double dist, double hum, final long temp_cal_info)
```

**备注：不同的SDK，该步骤的调用方法可能会有不同，具体请参考`libir_sample`中的示例。**

**【Note: Different SDKs may call this step differently. For details, please refer to the example in `libir_sample`.】**

## 4.2.3 Step3

调用函数`releaseTemperatureCorrection`释放测温二次修正资源

【Call the function `releaseTemperatureCorrection` to release the temperature measurement secondary correction resources】

```
/**
 * 释放测温二次修正资源【Release the resources for the second correction of temperature measurement】
 * release the resource called by {@link #readNucTableFromFlash()} and
 * {@link #temperatureCorrection(double, byte[], double, double, double, double, double, long)}
 *
 * @param temp_cal_info
 */
public void releaseTemperatureCorrection(final long temp_cal_info)
```

该函数只需要在退出的时候调用一次。

【This function only needs to be called once on exit.】

# 5 SDK中环境变量校正算法说明【SDK solution instruction】

## 5.1 Step1

调用函数read_nuc_parameter【Call the function read nuc parameter】

该函数的作用是读取模组中的测温参数，为测温修正提供测温映射数据。(大约需要10秒)。生产好的模组在高低增益下分别读取一次并存储即可。

【The function is to read temperature parameters in the module and provide temperature mapping data for correction. (It takes about 10 seconds) The produced modules can be read and stored at high and low gain respectively. 】

## 5.2 Step2

调用函数calculate_org_env_cali_parameter【Call the function calculate_org_env_cali_parameter】

该函数的功能是读取模组中的内置环境变量参数并进行相关系数计算。需要等待该函数返回正确的值之后再进行下一步。高低增益的内置环境变量参数可能不同，需要分别读取。如果内置环境变量参数未改变，则不需要每次都重新读取。

【The function reads the parameters of the built-in ambient variables in the module and calculates the coefficients. You need to wait for the function to return the correct value before proceeding to the next step. The parameters of the built-in ambient variables for high and low gain may be different and need to be read separately. If the built-in ambient variable parameters have not changed, you do not need to re-read each time. 】

## 5.3 Step3

调用函数calculate_new_env_cali_parameter【Call the function new_env_cali_parameter】

该函数的功能是根据设定的参数(反射温度，目标发射率，大气温度，距离，湿度)进行新的环境变量修正系数计算。如果设定参数没有改变，不需要每次都重新计算。高低增益可分别设置。

【The function is based on the set parameters (reflection temperature, emissivity, atmospheric temperature, distance, humidity) to perform a new ambient variable correction coefficient calculation. If the Settings are not changed, you do not need to recalculate each time. High and low gain can be set separately. 】

## 5.4 Step4

调用函数temp_calc_with_new_env_calibration【Call the functiontemp_calc_with_new_env_calibration】

该函数的功能是将传入的温度进行环境变量校正，然后输出校正后的温度。

【The function is to adjust the incoming temperature for the ambient variables and output the corrected temperature. 】

- 关机时nuc_table，nuc_factor，org_env_factor这些全局变量需要保存到文件中（高低增益分别存放），下次开机时根据增益状态从文件中读取这些参数并给这些变量赋值。然后直接进行第三步和第四步进行新的环境变量校正，输出校正后温度。

【During shutdown, nuc_table, nuc_factor, org_env_factor and other global variables need to be saved in files (high and low gains are stored separately). These parameters should be read from files and assigned to these variables according to the gain status when starting up next time. Then, the third and fourth steps are directly carried out for the correction of new ambient variables, and the corrected temperature is output.】

- 如果上位机重新设置过模组固件内置环境变量参数，需要调用第二步的函数重新计算固件中环境修正系数。

  【If the master terminal has reset the built-in ambient variable parameters of the module firmware, it is necessary to call the function in the second step to recalculate the ambient correction coefficient in the firmware.】

- 注意模组获取原始数据时，nuc_table的flash起始地址是0xda000，需要修改read_nuc_parameter函数中这两处spi_read函数的参数。参考cmd.cpp中的case 16和case 17。

  【Note that when the module obtains the original data, the start flash address of nuc_table is 0xda000. You need to modify the parameters of spi_read function in read_nuc_parameter function. Refer to case 16 and case 17 in cmd. cpp.】